Lesson End Project
Creating a Dynamic To-Do List Application

Project agenda: The goal is to develop a dynamic to-do list application using React, focusing on state management, event-driven updates, and rendering optimization. The application will support task addition, completion toggling, and removal, demonstrating React Hooks and efficient rendering techniques.

Description: You are tasked with building a React-based to-do list application to improve task management efficiency. This involves handling user input, updating state dynamically, and optimizing rendering performance. The project ensures smooth user interaction by leveraging React Hooks, event-driven updates, and lifecycle management.

Tools required: Visual Studio Code and Node.js

Prerequisites: None

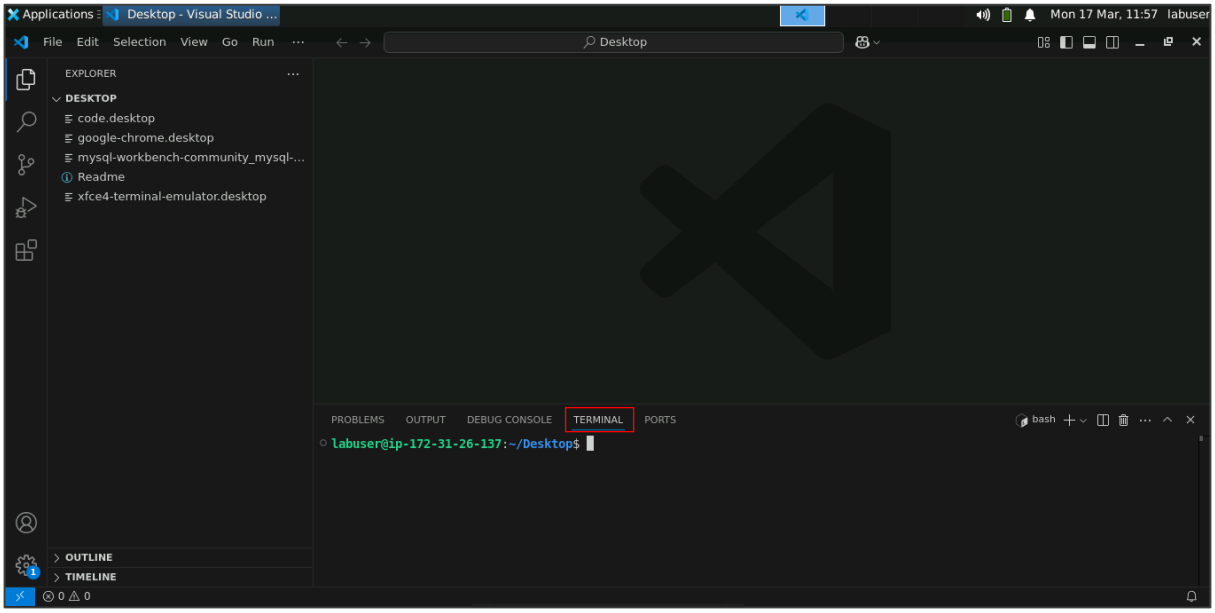
Expected deliverables: A fully functional React-based to-do list application with dynamic task management features. The application will support task addition, completion toggling, and deletion while ensuring optimized performance. It will include structured reusable components, state management with useState, and lifecycle management with useEffect.

Steps to be followed:

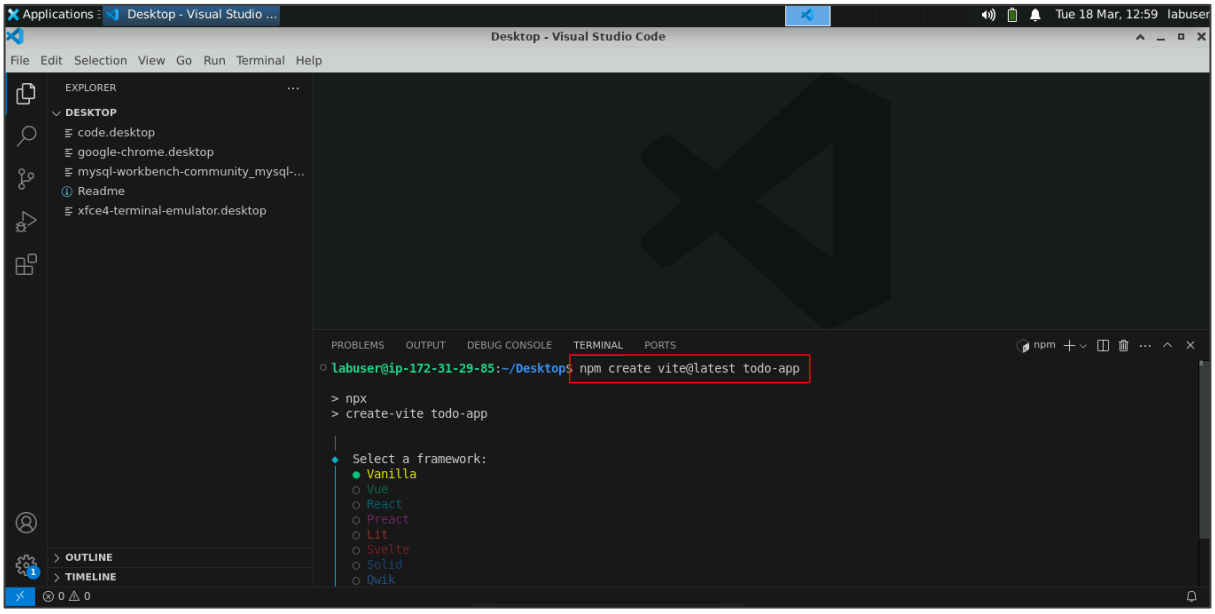
1. Set up a new React project using Vite
2. Implement core components
3. Integrate components in App.jsx
4. Style the application
5. Run and verify the application

Step 1: Set up a new React project using Vite

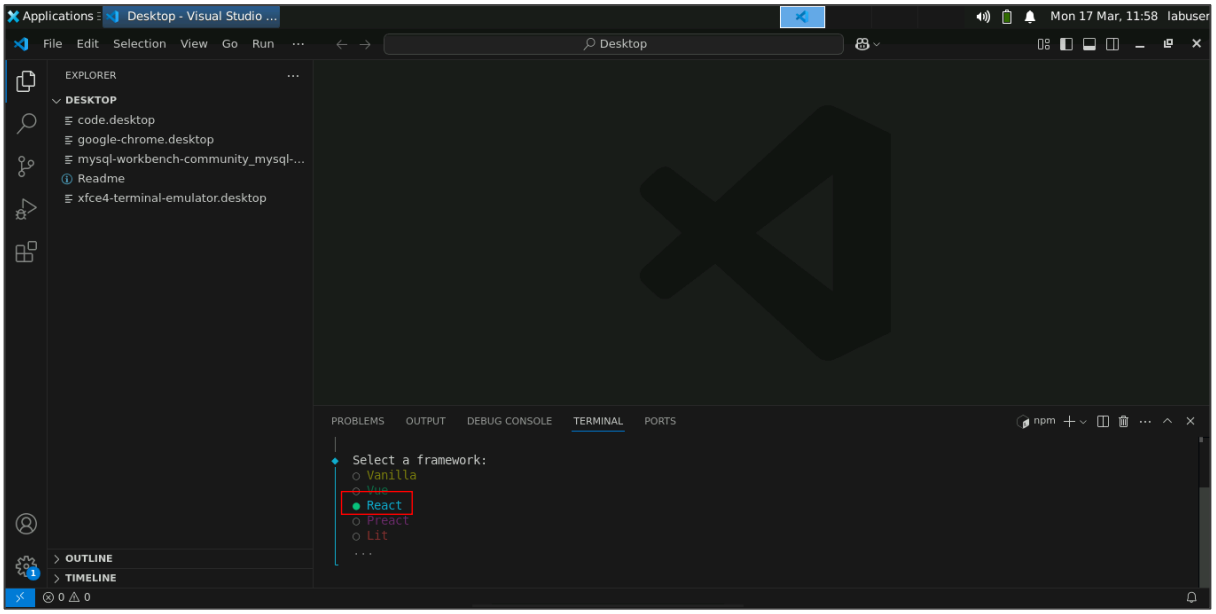
1.1 Navigate to VS Code and open the terminal



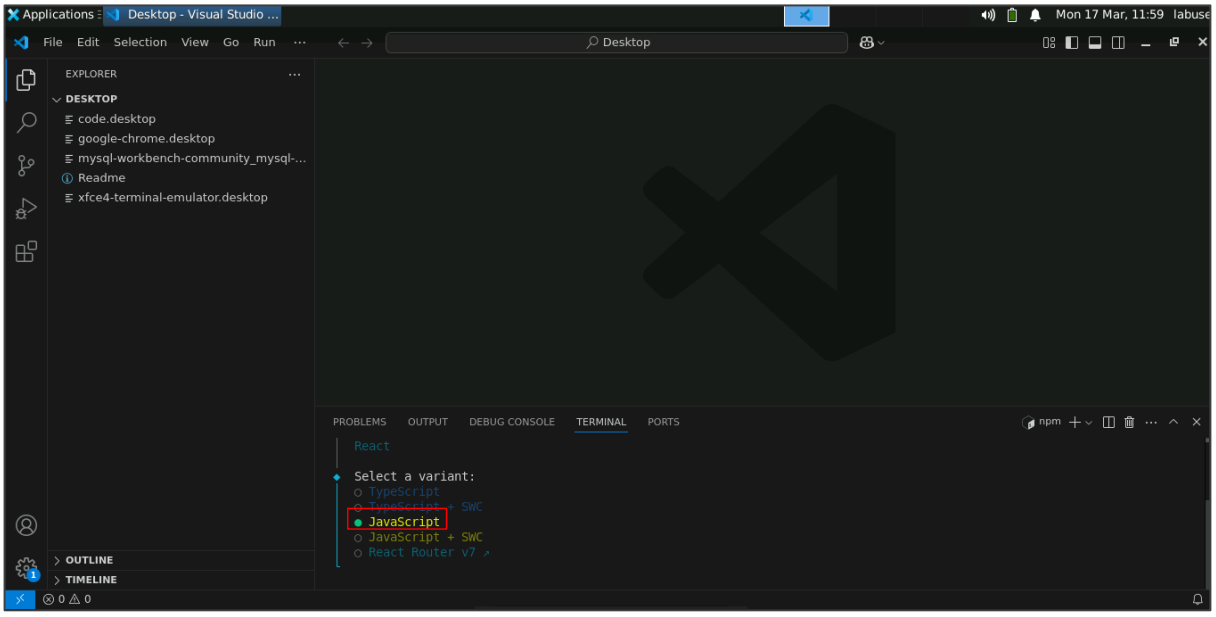
1.2 Run the following command to create a new React project using Vite:
npm create vite@latest todo-app



1.3 Select **React** and press enter

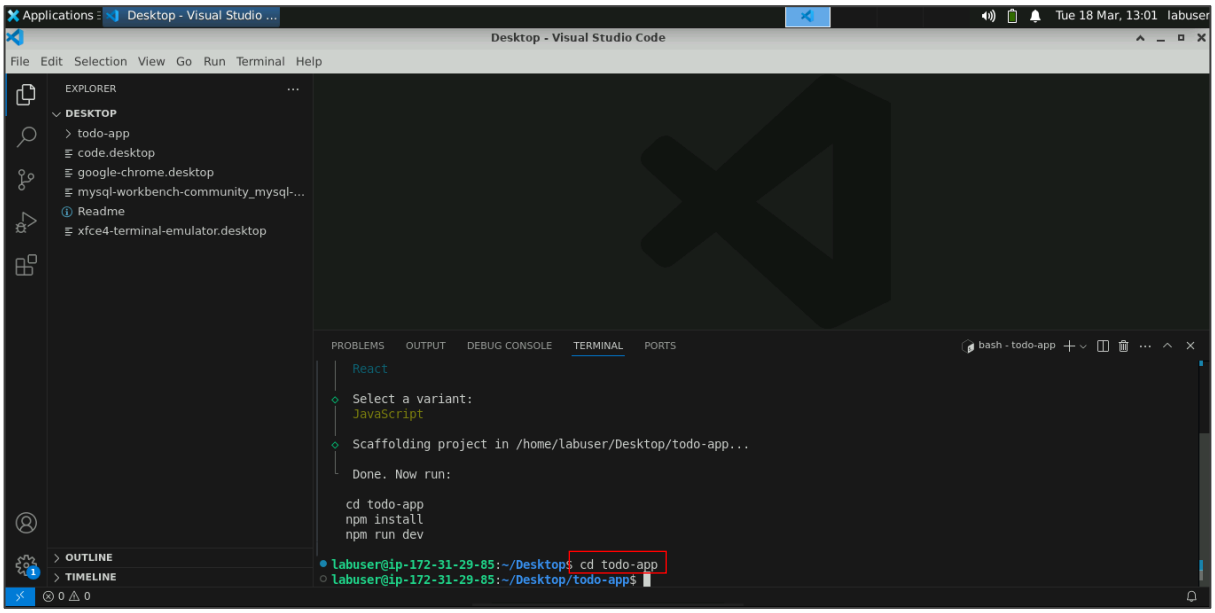


1.4 Select **JavaScript** and press enter



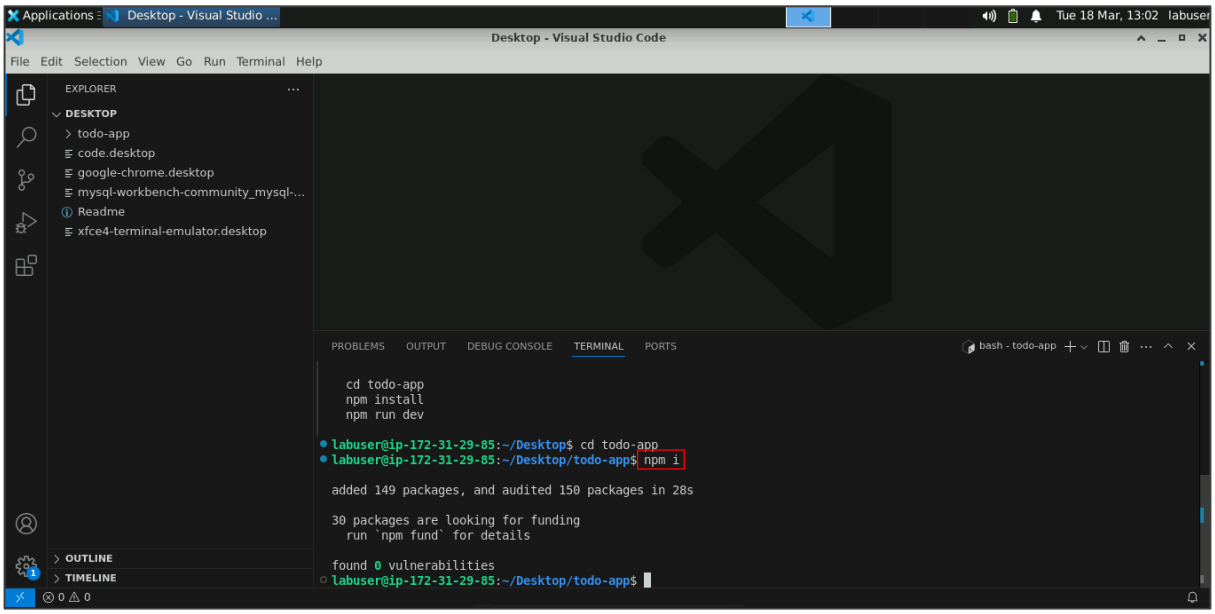
1.5 Run the following command to navigate to the project folder:

cd todo-app



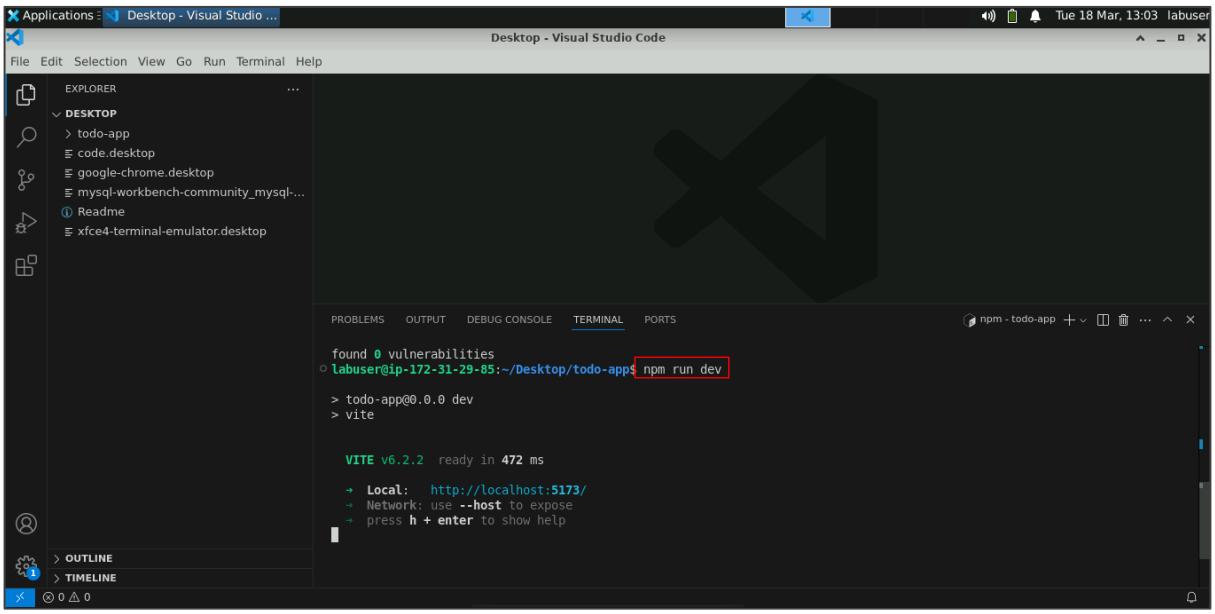
1.6 Run the following command to install dependencies:

npm i



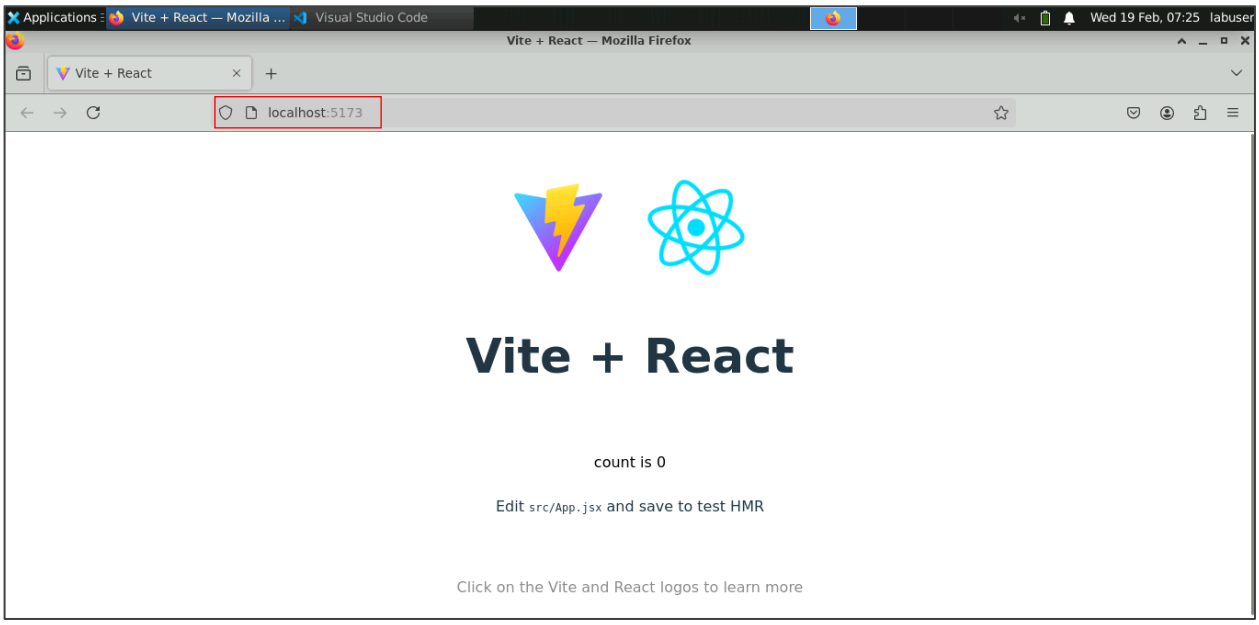
1.7 Run the following command to start the development server:

npm run dev



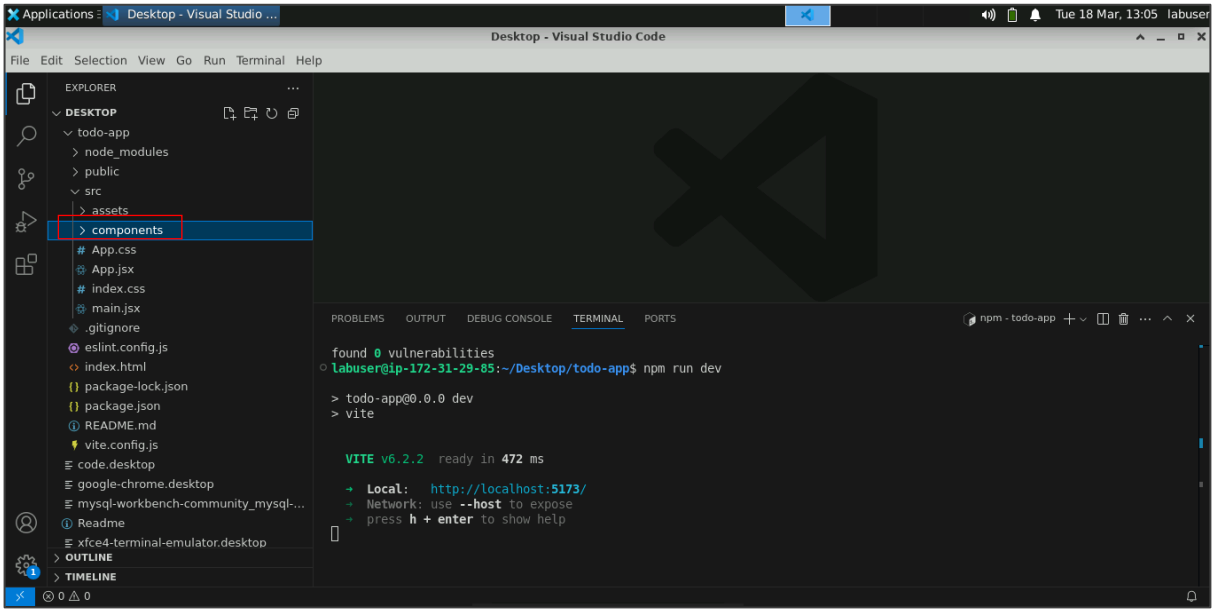
1.8 Navigate to the web browser and paste the given link:

http://localhost:5173

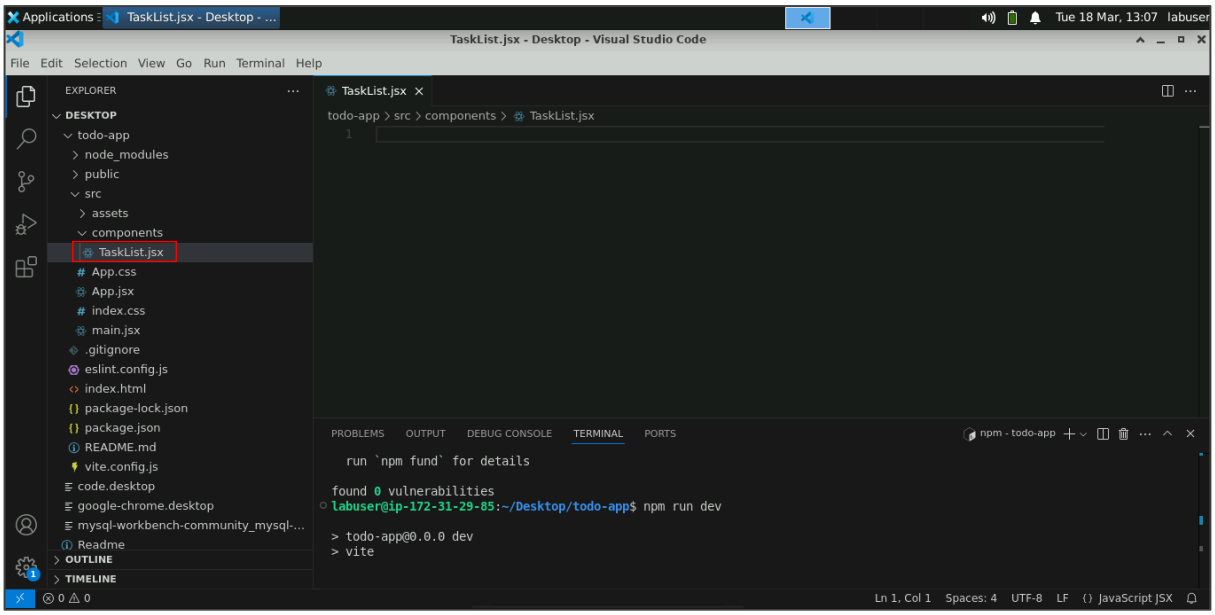


Step 2: Implement core components

2.1 Inside the **src** folder, create a new folder named **components**

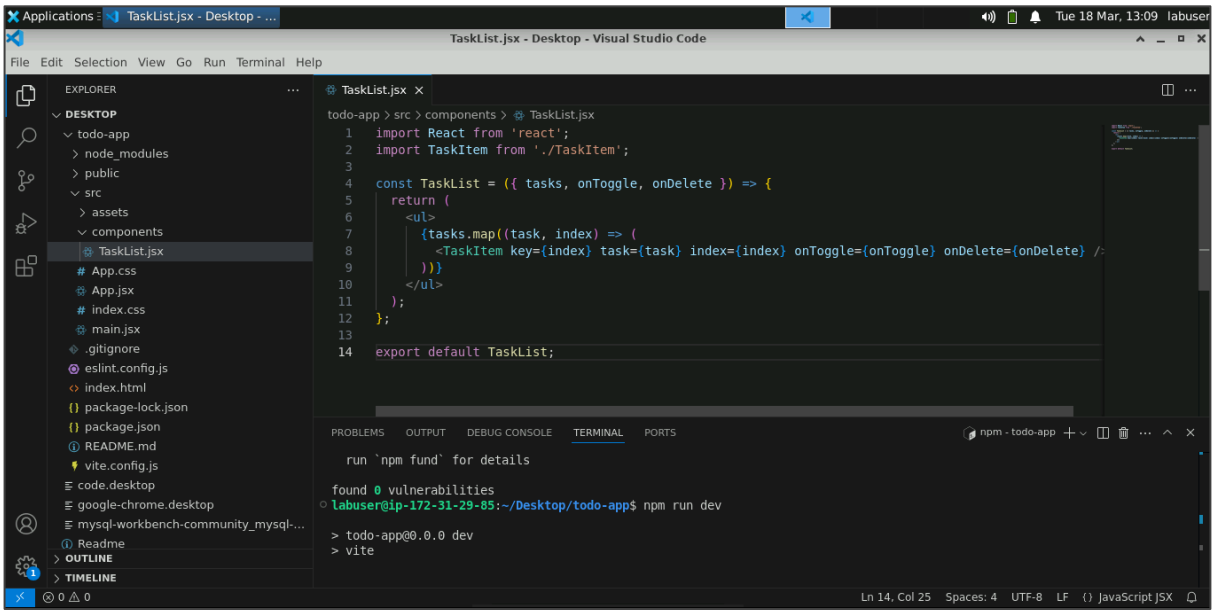


2.2 Inside the **components** folder, create a new file named **TaskList.jsx**

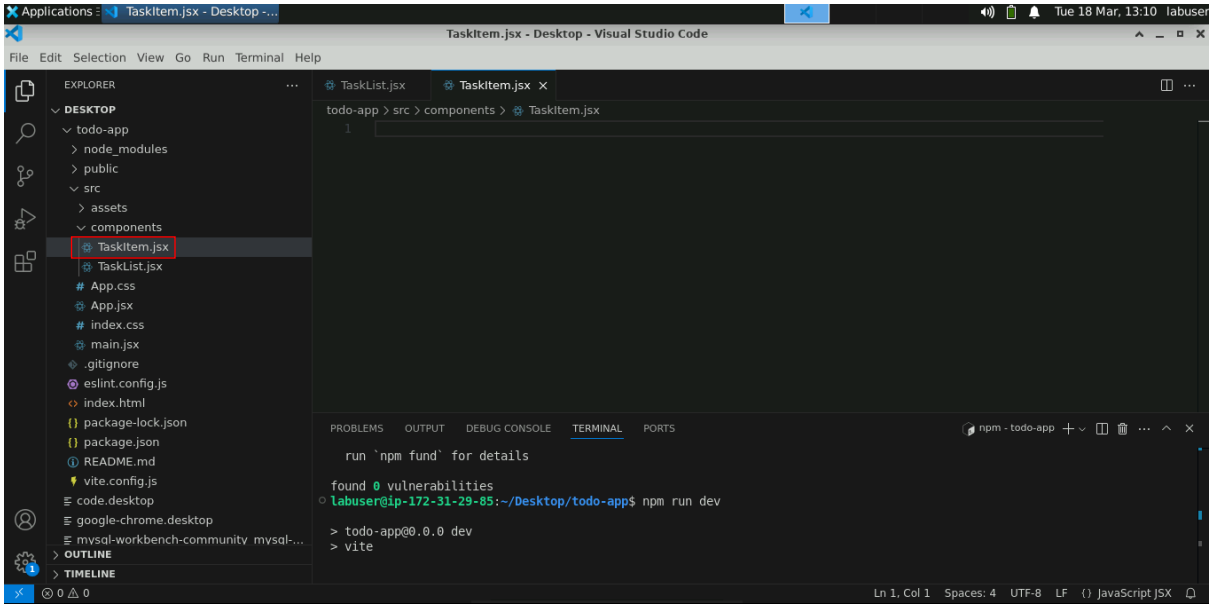


2.3 Add the following code to **TaskList.jsx:**

```
import React from 'react';
import TaskItem from './TaskItem';

const TaskList = ({ tasks, onToggle, onDelete }) => {
  return (
    <ul>
      {tasks.map((task, index) => (
        <TaskItem key={index} task={task} index={index} onToggle={onToggle}
onDelete={onDelete} />
      ))}
    </ul>
  );
};

export default TaskList;
```

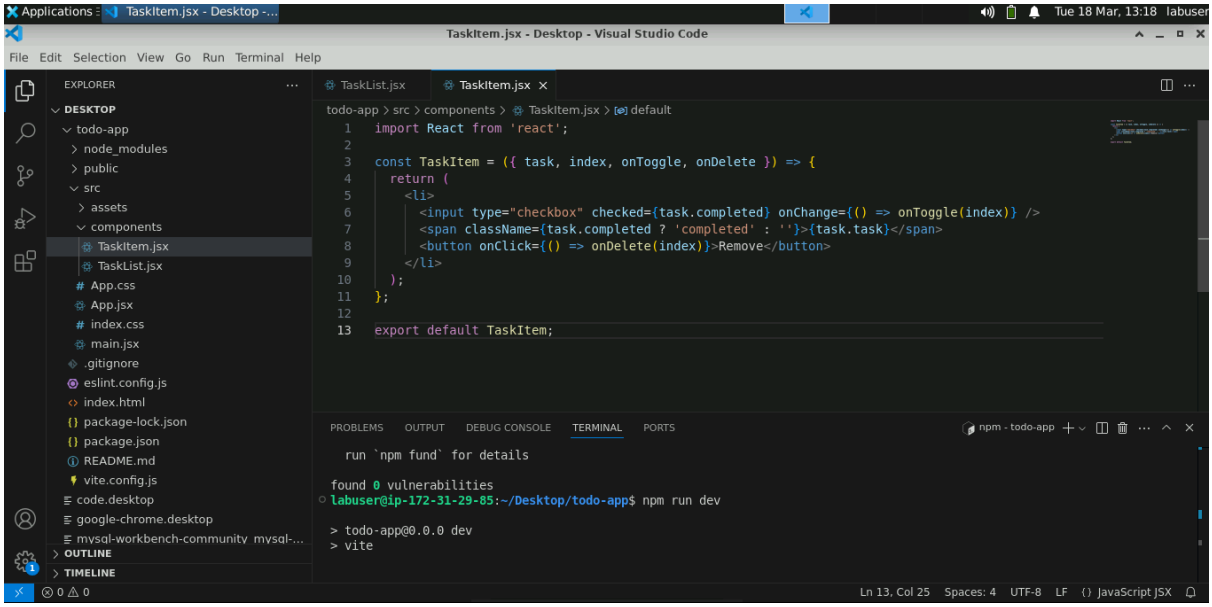


2.4 Inside the **components** folder, create a new file named **TaskItem.jsx**

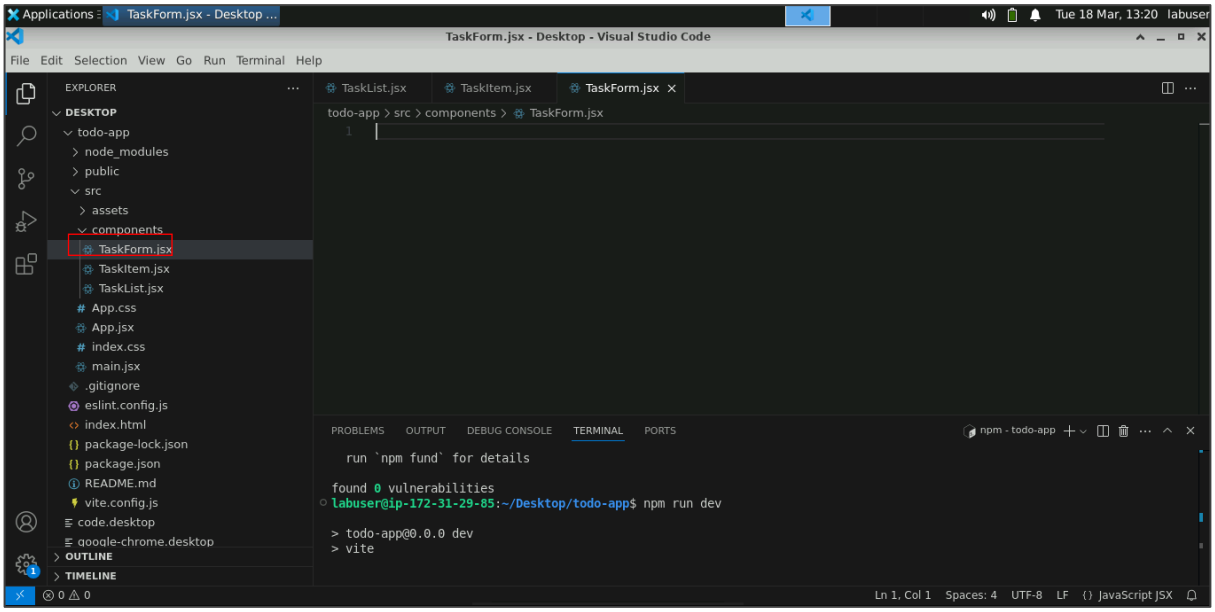


2.5 Add the following code to **TaskItem.jsx:**

```
import React from 'react';

const TaskItem = ({ task, index, onToggle, onDelete }) => {
  return (
    <li>
      <input type="checkbox" checked={task.completed} onChange={() =>
onToggle(index)} />
      <span className={task.completed ? 'completed' : ''}>{task.task}</span>
      <button onClick={() => onDelete(index)}>Remove</button>
    </li>
  );
};

export default TaskItem;
```

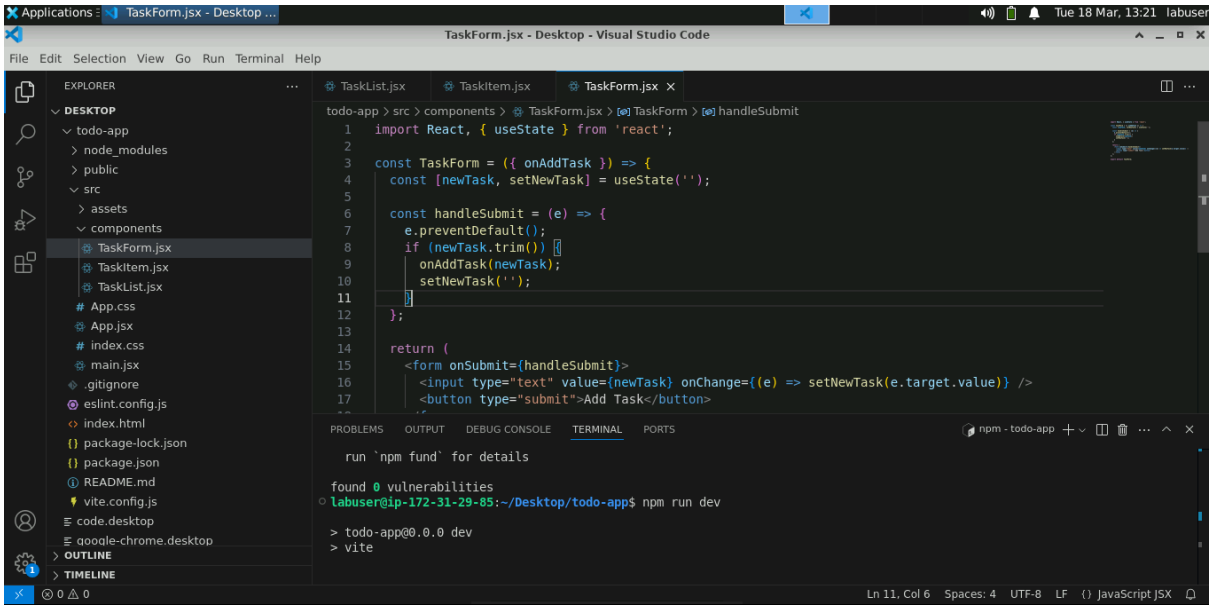


2.6 Inside the **components** folder, create a new file named **TaskForm.jsx**



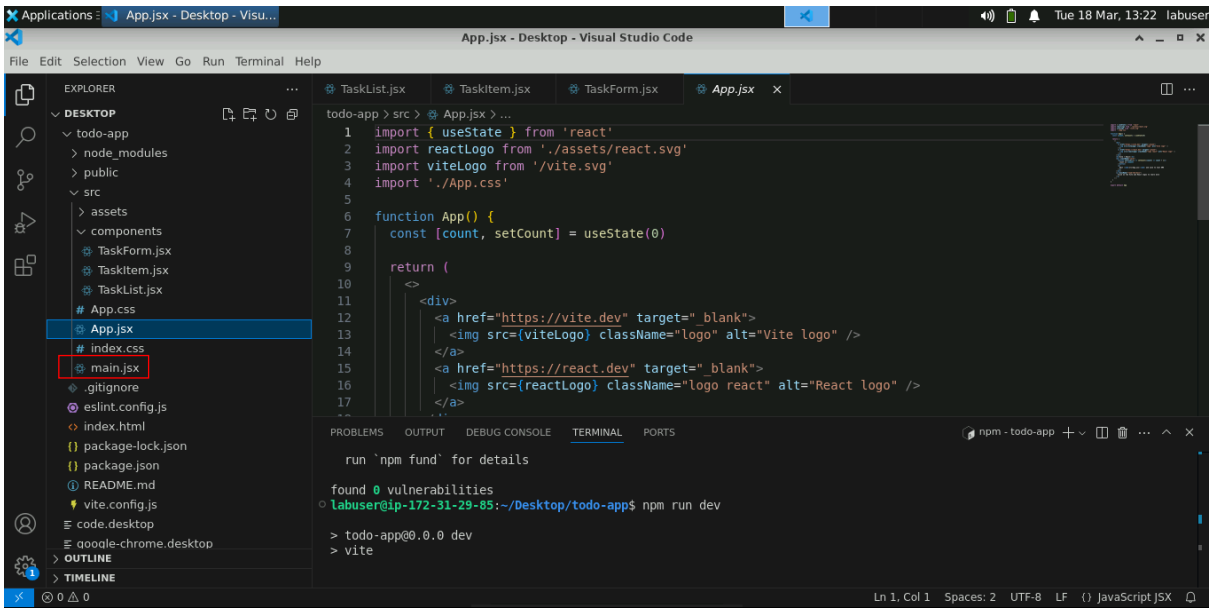
2.7 Add the following code to **TaskForm.jsx:**

```
import React, { useState } from 'react';

const TaskForm = ({ onAddTask }) => {
  const [newTask, setNewTask] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    if (newTask.trim()) {
      onAddTask(newTask);
      setNewTask('');
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" value={newTask} onChange={(e) =>
setNewTask(e.target.value)} />
      <button type="submit">Add Task</button>
    </form>
  );
};

export default TaskForm;
```



Step 3: Integrate components in App.jsx

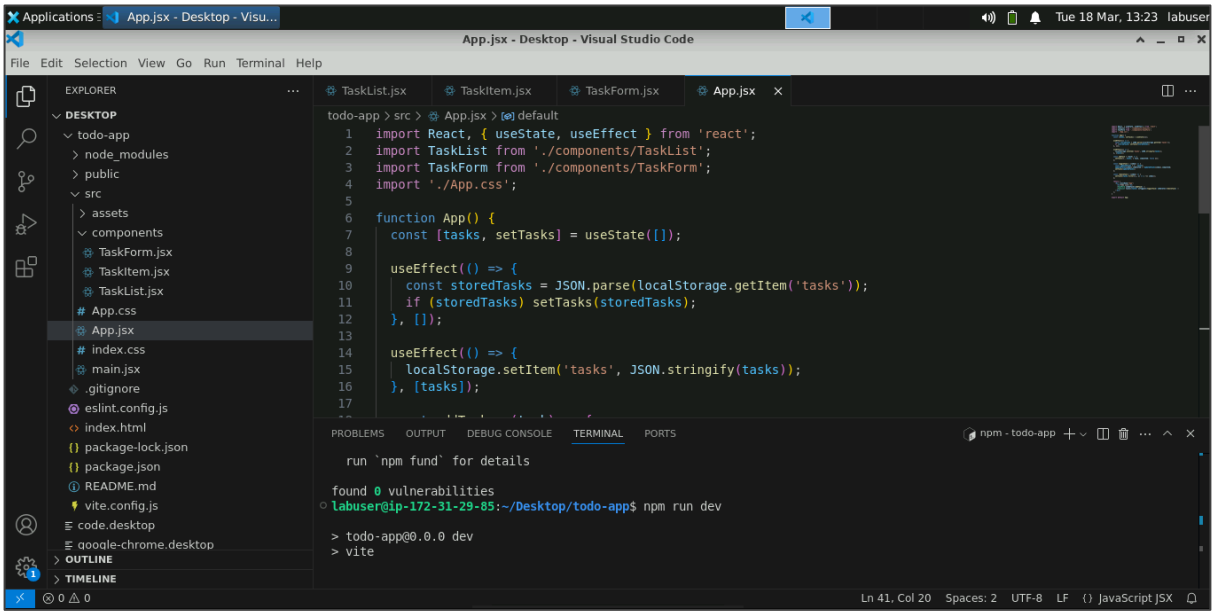
3.1 Open the **App.jsx** file inside the **src** folder



3.2 Paste the following code to include the new components in **App.jsx**:

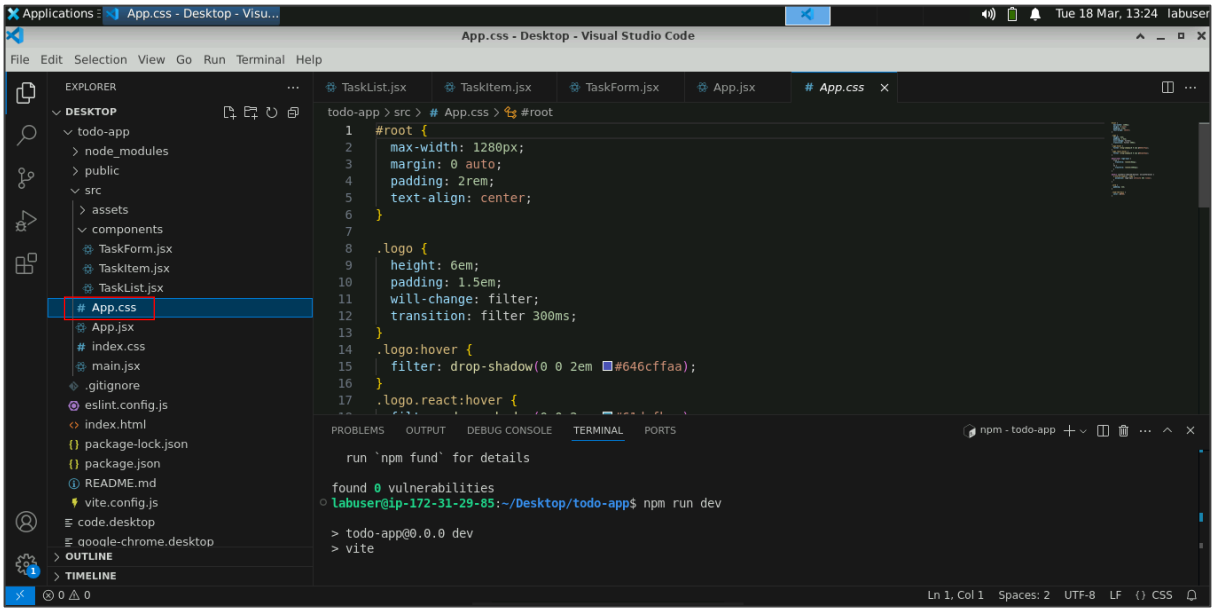
```
import React, { useState, useEffect } from 'react';
import TaskList from './components/TaskList';
import TaskForm from './components/TaskForm';
import './App.css';

function App() {
  const [tasks, setTasks] = useState([]);

  useEffect(() => {
    const storedTasks = JSON.parse(localStorage.getItem('tasks'));
    if (storedTasks) setTasks(storedTasks);
  }, []);

  useEffect(() => {
    localStorage.setItem('tasks', JSON.stringify(tasks));
  }, [tasks]);

  const addTask = (task) => {
    setTasks([...tasks, { task, completed: false }]);
  };

  const toggleTask = (index) => {
    const updatedTasks = [...tasks];
    updatedTasks[index].completed = !updatedTasks[index].completed;
```

```
  setTasks(updatedTasks);
 };

 const removeTask = (index) => {
  setTasks(tasks.filter((_, i) => i !== index));
 };

 return (
  <div className="App">
   <h1>Todo List</h1>
   <TaskForm onAddTask={addTask} />
   <TaskList tasks={tasks} onToggle={toggleTask} onDelete={removeTask} />
  </div>
 );
}

export default App;
```



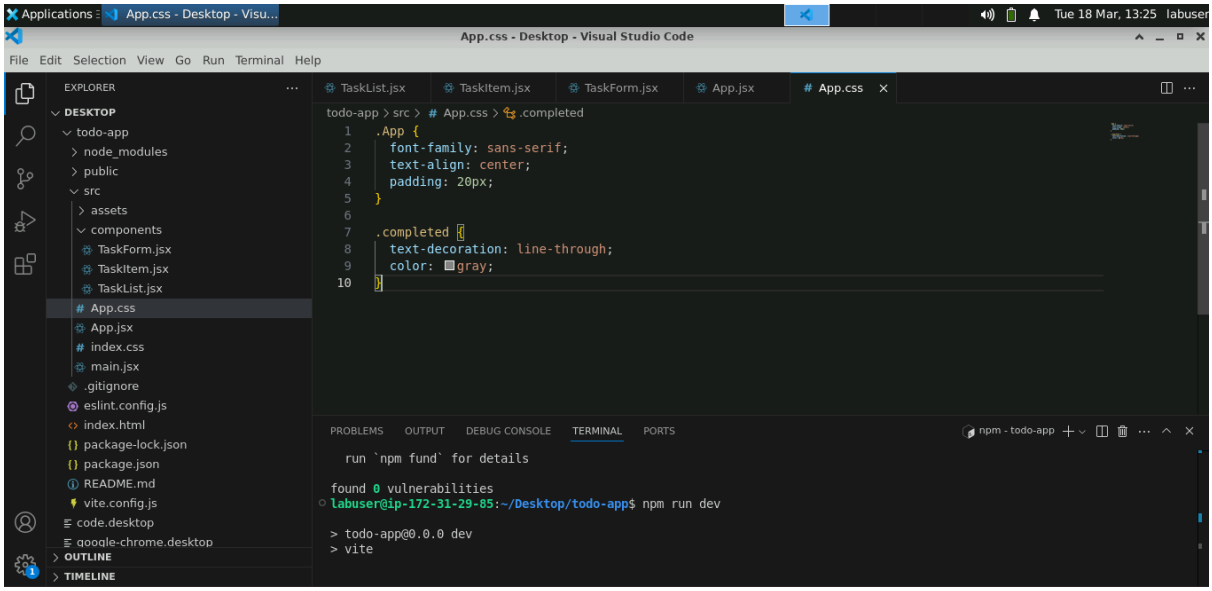
Step 4: Style the application

4.1 Open the **App.css** file inside the **src** folder



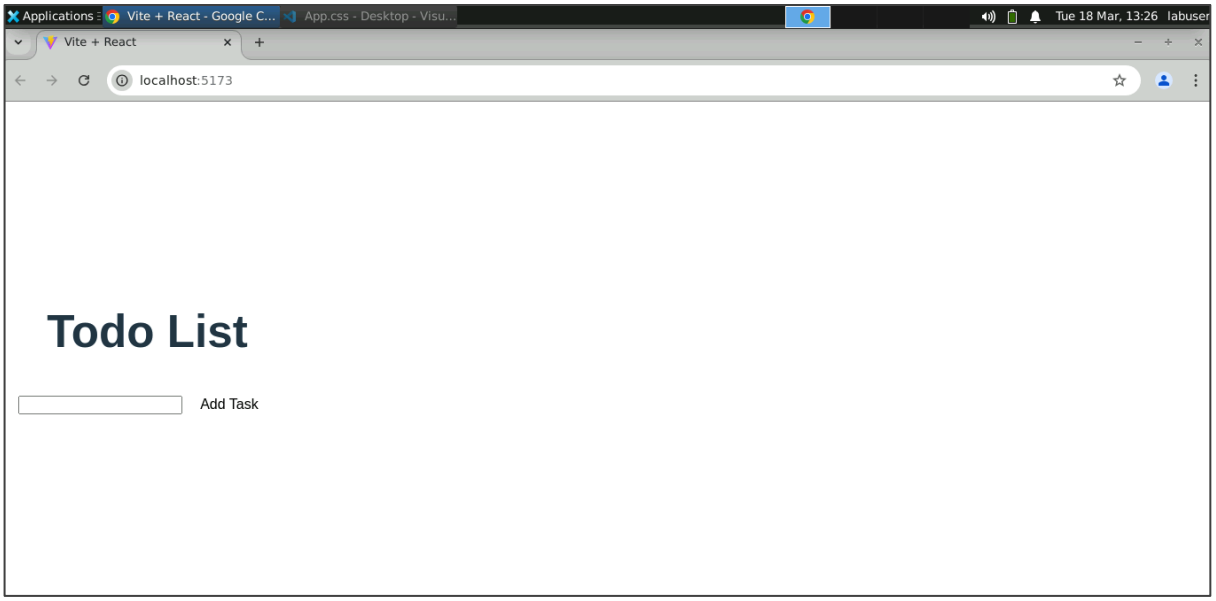
4.2 Paste the following code to include the new components in **App.css**:

```
.App {
  font-family: sans-serif;
  text-align: center;
  padding: 20px;
}

.completed {
  text-decoration: line-through;
  color: gray;
}
```

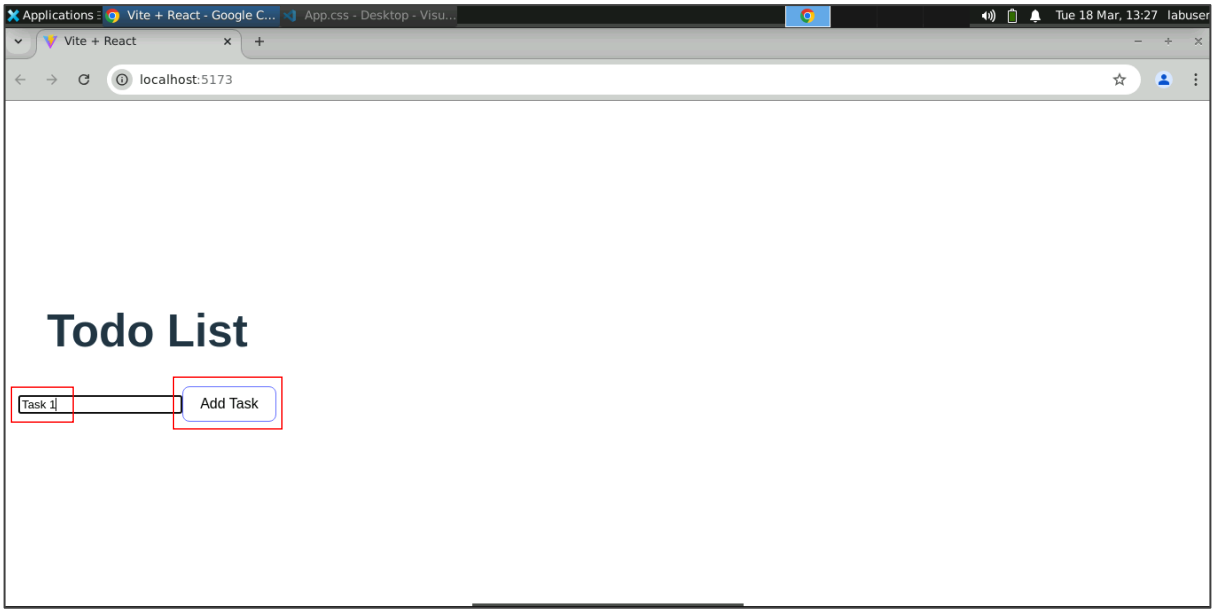


Step 5: Run and verify the application

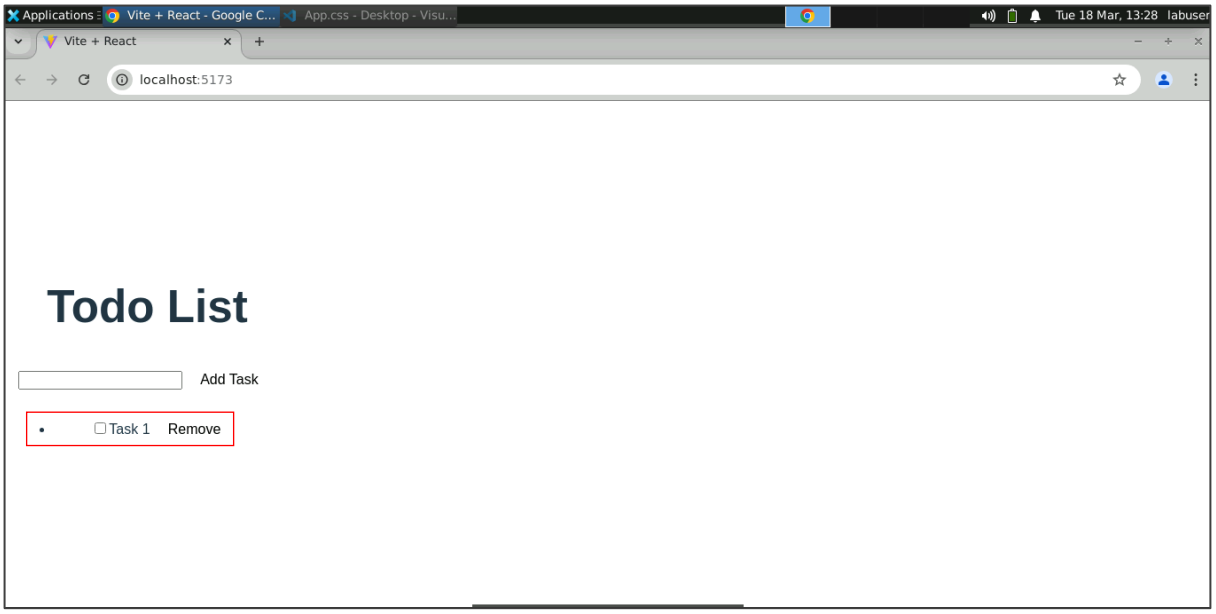
5.1 Navigate to the browser and refresh the webpage



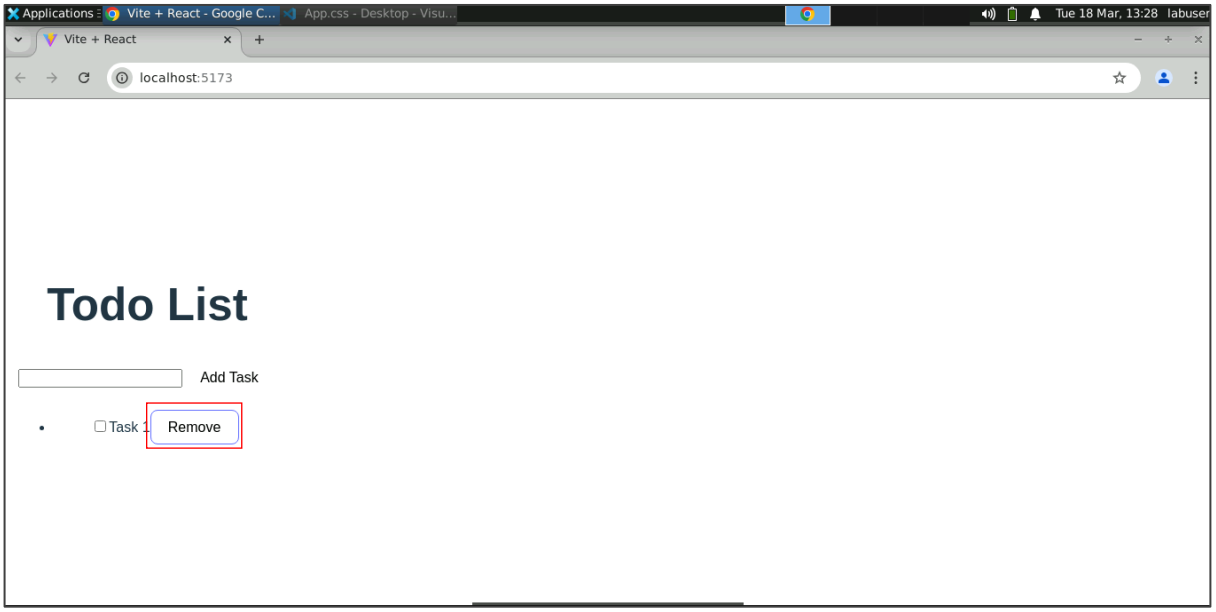
5.2 Enter your preferred task and click on **Add Task**



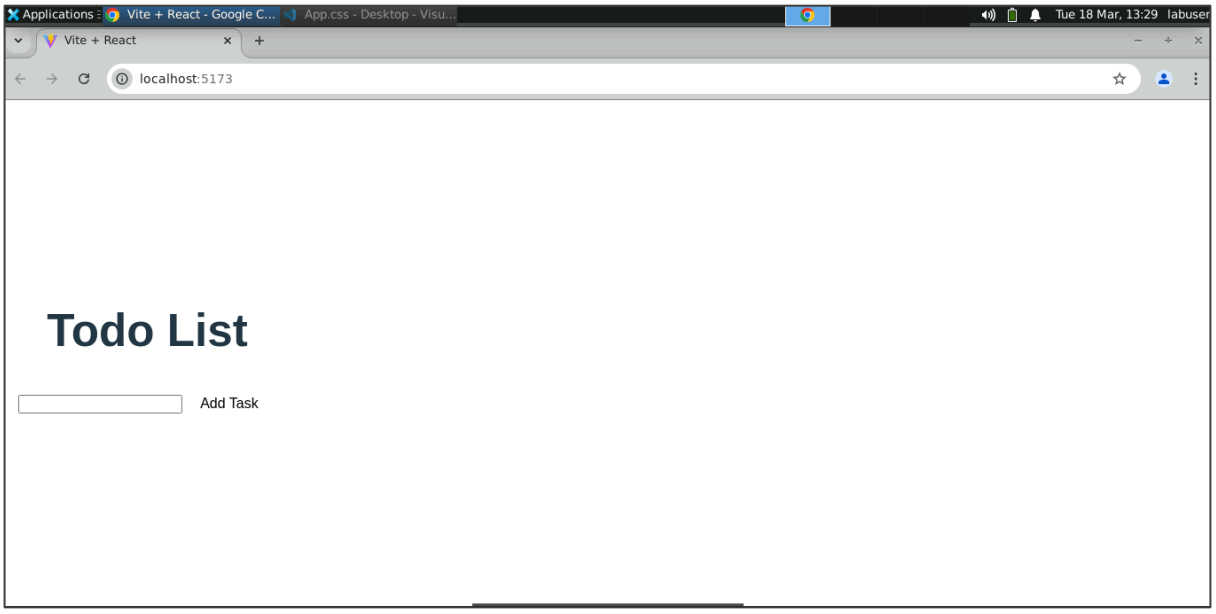
You will see a new Task added to the list.



5.3 Click on **Remove** to delete the task that was created



You will see a task deleted from the list.



By following the above steps, you have successfully implemented a fully functional React-based to-do list application. This project demonstrates React Hooks, state management, lifecycle methods, and event-driven updates. The efficient structure ensures scalability and a smooth user experience.